

CPEN 455 - Final Project - Conditional PixelCNN

Alexander Lassooij
Electrical & Computer Engineering
The University of British Columbia
alasso01@student.ubc.ca

Abstract

The report discusses the architecture, methods and experiments used to enhance a PixelCNN model with FiLM-based class conditioning, identifying an optimal configuration that balances generation quality and classification accuracy.

Model

The suggested model's implementation mainly preserved the existing U-Net-like base architecture provided. Modifications were made to incorporate class embeddings in order to output different probability distributions conditioned on a particular class. Class conditioning was added through FiLM (Featurewise Linear Modulation) layers. The FiLM layers were added within each gated ResNet block and to the output head (see Fig. 1).

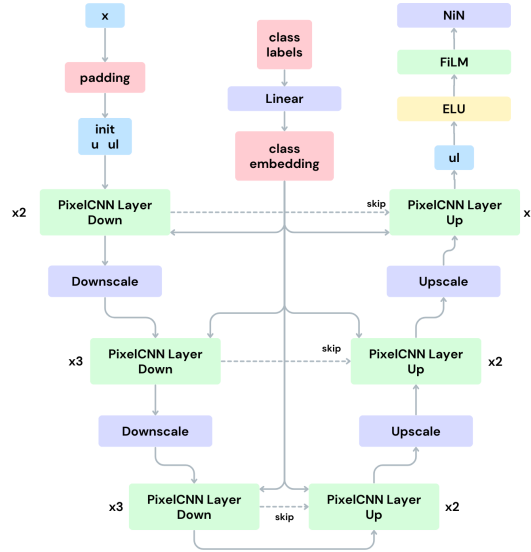


Figure 1: High-level computational graph of the Conditional PixelCNN model.

Models & Layers

Convolutional Layers

Vertical Streams (down_shifted_conv2d, down_shifted_deconv2d)

These layers process existing information from the rows above. This is achieved with `nn.ZeroPad2d` which shifts the receptive field downward. This way, the pixels in the bottom are effectively masked.

Both use 2x3 kernels compared to the 2x2 kernels of the vertical streams in order to widen the receptive field to capture more context. Additionally, both use [normalization] for stability.

Horizontal Streams (`down_right_shifted_conv2d`, `down_right_shifted_deconv2d`)

In contrast to the vertical streams, these convolutional layers shift the receptive field downward and to the right, masking the pixels to the bottom and the right.

The convolutions in the up pass (encoder) produce a compressed high-level feature representation, while the transposed convolutions in the down pass (decoder) reconstruct pixel values as mixtures of discretized logistic distributions. These convolutional layers are used in the ResNet layers as well as for down-/upscaling with stride=2 to shrink and expand the size of the feature representations.

Gated ResNet (`gated_resnet`)

The GatedResNet block is the core of the PixelCNN architecture. It begins with a masked convolution, followed by GroupNorm and a concatenated ELU that doubles feature dimensions. If enabled, skip connections are integrated via a 1x1 convolution. A second masked convolution and normalization lead into a gating mechanism where half the channels modulate the other half via a sigmoid (gate). A residual connection then adds the input back, improving gradient flow and learning:

$$\text{Gated}(x) = a \cdot \sigma(b) \quad a, b = \text{split}(x) \quad \text{out} = x_{\text{original}} + \text{Gated}(x) \quad (1)$$

FiLM

A method known as Featurewise Linear Modulation (FiLM) was used to add class conditioning to the model. On a high level, it performs a feature-wise affine transformation on a neural network's intermediate features by applying a linear scale and shift through 2 learned parameters, γ and β .

$$\text{FiLM}(F|\gamma, \beta) = \gamma F + \beta \quad (2)$$

The parameters were learned by passing the class embeddings through a small network (Fig. 2.a) consisting of two FC layers, layer normalization and ReLU activations. The hidden dimension of the FC layers was limited to 32 to avoid overfitting. The initial weights were set with Xavier initialization, while a learnable scaler a was initialized with 3.0. This scaler was added to strengthen the conditioning signal in each gated residual block, giving us:

$$\text{FiLM}(F|a, \gamma, \beta) = a\gamma F + \beta \quad (3)$$

The FiLM layers were added before the dropout layer of the gated residual blocks (see Fig. 2.b) and after the ELU activation at the output head. This allows the model to learn class-dependent representations in order to conditionally produce logistic distributions. This also enables classification based on the negative log-likelihood of an image given the predicted distribution, and for conditional sampling given a particular class embedding.

Discretized Mixture of Logistics

The Discretized Mixture of Logistics models pixel intensities as a mixture of logistic distributions, computing pixel likelihoods using CDFs between bin edges and aggregating them via log-sum-exp. The total loss is then the negative log-likelihood summed over all pixels [2, 3]. Each pixel's distribution is parameterized by a mean, scale, and coefficient:

$$P(x|\pi, \mu, s) = \sum_{i=1}^K \pi_i [\sigma((x + 0.5 - \mu_i)/s_i) - \sigma((x - 0.5 - \mu_i)/s_i)] \quad (4)$$

These parameters scale with the number of logistic components, which enhances the model's ability to represent complex pixel distributions but can lead to overfitting. Ultimately, it measures how likely each pixel is under the learned mixture model.

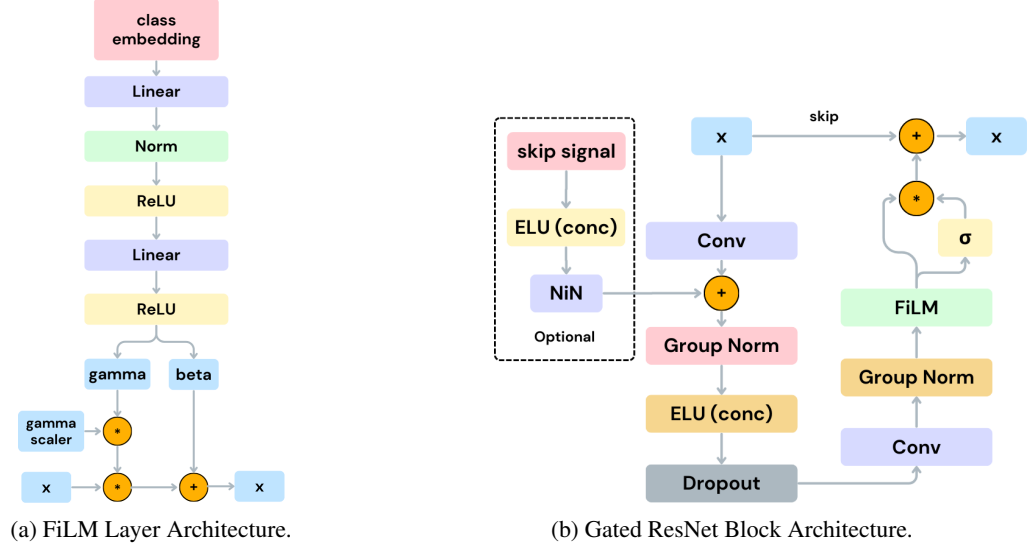


Figure 2

Experiments

Many different experiments were carried out to test their effect on the model’s ability to predict classify images and generate class-diverse samples. The model from Trial #9 struck a very good balance between validation accuracy and FID, and hence served as a reference point to perturb from in the following trials.

Ablation Study and Analysis

Model Size (nr_filters, nr_log_mix, nr_resnet)

Experimented with $\#filters \in [64, 120]$, $\#log\ mix \in [10, 50]$, $\#resnet \in [2, 4]$. While increasing these parameters increase the model’s capacity to model pixel distributions and extract better feature maps, this often lead to overfitting and bad generalization. At the same time, making the model larger helped estimate the distribution of the pixel better, improving FID. Increasing the number of ResNet blocks significantly slowed down training time as well. Therefore, a balance between model complexity and ability to generalize was needed, which was found to be at $\#filters \sim 80$, $\#log\ mix = 20$ and $\#resnet = 2$.

Fusion Method

The first attempt at fusion to condition the model was through concatenation of the class embeddings to the input (early fusion) and within the ResNet layers. This resulted in poor FID that would often spike up, indicating the this method of fusion might have been distorting spatial correlations. While removing fusion from the input helped, the FID was still unsatisfactory.

Using featurewise linear modulation (FiLM) stabilized the FID while maintaining a good validation accuracy. In later iterations of the FiLM layer’s design, the FID was also much lower and remained within a range of [20,30]. Adding a FiLM layer to the output head also slightly improved the FID. Middle fusion was also attempted by “FiLM-ing” the output of the encoder, but showed no improvement in accuracy and deteriorated the sample quality.

Modifying FiLM layer

Initially, a simple FiLM layer with a hidden layer was added after the dropout step in the ResNet layer to learn the parameters γ and β to apply class-dependent scale and shift. As aforementioned, this made the FID more stable, but both the FID and validation accuracy were low. We suspected that the conditioning signal was too weak, leading to poor class diversity and outputs that were a ‘mixture’ of the four classes. A learnable scaler was added to amplify the conditioning signal, which significantly improved results.

83 However, the FID had a tendency to deteriorate in later epochs. As observed, groups of pixels specific
 84 to a class were often over-saturated. A few methods were tested to address this, including : 1) a skip
 85 connection in the FiLM layer itself; 2) TanH activation instead of ReLU at the head of the FiLM layer,
 86 3) a lower value for the initial gamma scaler α (2.0 instead of 3.0), 4) adding FiLM before dropout
 87 and 5) adding FiLM after the second convolution to emulate the implementation in Fig.3 of [1].

88 Temperature

89 A small experiment was conducted to add temperature to the log scales of the discretized mix of
 90 logits. We experimented with a low temperature of $t = 0.8$ and $t = 1.2$. The low temperature had a
 91 negative effect as it reduced the diversity of the samples, while the higher temperature lead to a slight
 92 oversaturation of pixels.

93 Optimizer

94 The original Adam optimizer was replaced with an AdamW optimizer in an attempt to stabilize the gra-
 95 dient, since AdamW updates the parameters with decoupled weight decay, helping to prevent overfit-
 96 ting and improve generalization. A standard weight decay rate of $1e-5$ was used. The initial learning
 97 rate of $\eta = 0.0002$ was kept, while various rates of LR decay between $[0.9995, 0.99995, 0.999995]$
 98 were tested. The LR decay did not show a significant change in performance. Another attempt to
 99 fix over-saturated pixels was to use gradient clipping, but this lead to very under-saturated, “dark”
 100 samples.

101 Training Epochs

102 The training epochs were limited to 200-250 epochs, since it was determined early on that the model
 103 tends to severely overfit past 300 epochs. The rate of decrease of BPD also greatly diminishes after
 104 250 epochs.

105 As we can see from the experiment results in Tables 1 and 2, trial #9 - a very early experiment -
 106 showed to be the best configuration. It strikes a very good balance between a moderate model size to
 107 mitigate overfitting, while still being powerful enough to learn rich, class-conditional representations
 108 and generate visually diverse and semantically accurate samples.

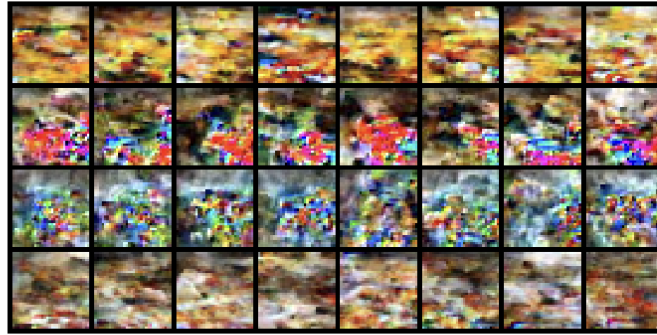


Figure 3: Samples generated after epoch 200 by model of Trial #9. Note how class-specific areas are slightly over-saturated.

109 Conclusion

110 Despite extensive experimentation and modifications, we were not able to improve upon this model
 111 in terms of both FID and classification accuracy, making it the most reliable and balanced setup for
 112 our task.

113 Although Trial #9 remained the strongest configuration, the experimentation process offered valuable
 114 insights into the model’s behavior. It allowed us to learn about the impact of conditioning strength,
 115 FiLM placement, architectural choices, deepening our understanding of this class of models.

116 For future work, it would be worth exploring better architecture for FiLM conditioning and alternative
 117 conditioning methods—such as attention-based conditioning — to provide more stable and expressive
 118 control.

References

- [1] Perez, E., Strub, F., Harm, D. V., Dumoulin, V., & Courville, A. (2017, September 22). FiLM: Visual Reasoning with a General Conditioning Layer. arXiv.org. <https://arxiv.org/abs/1709.07871>
- [2] Salimans, T., Karpathy, A., Chen, X., & Kingma, D. P. (2017, January 19). PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications. arXiv.org. <https://arxiv.org/abs/1701.05517>
- [3] Van Den Oord, A., Kalchbrenner, N., Vinyals, O., Espeholt, L., Graves, A., & Kavukcuoglu, K. (2016, June 16). Conditional Image Generation with PixelCNN Decoders. arXiv.org. <https://arxiv.org/abs/1606.05328>

Appendix

Trial #	Model Design Notes	#log mix	#filters	#resnet	decay	epochs	warmup	emb dim
1	No cond., Cosine Decay	10	100	4	N/A	500	150	N/A
2	No cond., Step LR Scheduler	15	100	3	0.9995	350	150	N/A
3	Early and ResNet Fusion with Concat	50	120	2	0.9995	300	50	64
4	ResNet Fusion with Concat	50	120	2	0.9995	300	50	64
5	ResNet Fusion with FiLM	50	120	2	0.99995	200	50	64
6	ResNet Fusion with FiLM	10	64	2	0.999995	200	50	64
7	Late Fusion with FiLM, fixed scaling	10	64	2	0.999995	350	50	16
8	Same but larger model	10	80	3	0.99995	200	50	16
9	Learned scale γ	20	80	2	0.99995	200	50	16
10	Larger model	20	120	3	0.9995	225	50	32
11	Large model, 2 Res-Blocks	30	90	2	0.9995	200	50	32
12	TanH in FiLM	30	90	3	0.9995	225	50	32
13	AdamW optimizer	20	120	2	0.999995	200	50	16
14	Middle fusion with FiLM	20	80	2	0.999995	225	25	16
15	FiLM before dropout	20	80	2	0.99995	200	50	16
16	Continued Trial #9	-	-	-	-	-	-	-
17	Lower temperature (0.8)	20	80	2	0.99995	200	50	16
18	Higher temperature (1.2)	20	80	2	0.99995	200	50	16
19	FiLM after 2nd conv	20	80	2	0.99995	200	50	16
20	Reduced gamma scale, FiLM skip connection	20	80	2	0.99995	200	50	16
21	Gradient Clipping	20	80	2	0.99995	200	25	16

Table 1: Experiment Trial Setup

Trial #	Notes	FID	BPD	Val Acc (%)	Test Acc (%)
1	FID shoots up at end	60	2.82	N/A	N/A
2	Better FID, not conditioned	31.7	2.83	N/A	N/A
3	FID exploded	>300	3.84	52.3	N/A
4	Good val acc, FID spikes	>100	2.92	70.6	N/A
5	Best val acc, bad FID	58	2.80	87.6	N/A
6	Lower val acc, better FID	45	3.10	80.8	N/A
7	Very good early FID	67	2.94	76.4	N/A
8	FID more stable	28.3	3.10	62.3	N/A
9	Best FID + accuracy balance	20.3	3.07	82.4	78.5
10	Worse val acc and FID	43.3	2.87	71.3	N/A
11	Good FID, low val acc	27.8	3.03	59.3	N/A
12	Slightly worse val acc	29.9	3.09	68.3	N/A
13	Strong FID, weak val acc	22.8	3.08	76.5	68.5
14	No improvement	39.4	3.07	73.2	N/A
15	No improvement	30.2	3.05	68.7	N/A
16	Model overfit	49	2.90	60.7	N/A
17	Poor class diversity	35.1	3.06	72.7	N/A
18	Over saturation	30.6	3.04	72.5	N/A
19	Very strong over saturation	32.6	3.00	77.9	N/A
20	Weak class conditioning	36.4	2.94	71.2	N/A
21	Gradient Clipping	24.85	3.66	80.4	74.9

Table 2: Experiment Trial Results. Note : all reported metrics from final epoch in training. Only models with decent validation accuracy were subject to the test set. The batch size (32) and learning rate (0.0002) were kept constant, so they were omitted from this table.